

PB138 — Web Technologies

Seminar 01: Introduction

What is this course about?

- An **introductory course** to web development
- We cover many technologies — breadth over depth
- Goal: build something **real and practical**
- Emphasis on **hands-on skills**, collaboration, and AI tools
- Tech stack: **React 19**, Tailwind CSS, TypeScript, Tanstack, Drizzle, and more

Semester overview

Week	Topic
01	Introduction, setup, React basics
02	React, Tailwind CSS, Shadcn
03	TypeScript
04	REST API + Codegen
05	DB, modeling, Drizzle
06	Tanstack Router
07	React Hook Form
08	Agentic Development
09	AuthZ, RBAC
10	Pipelines

Course Evaluation

Point sources

- **Project** — up to **70 points**
- **Exam** — up to **30 points**
- **Total: 100 points**

Bonus points may be awarded for outstanding work and active participation.

Grading scale

Grade	Points
A	94+
B	88–93
C	82–87
D	76–81
E	70–75
F	below 70

For credit-only enrollment: **60+ points** required.

Exam

- Format: **quiz / questionnaire** on web development topics
- Up to **30 points**
- You get up to **3 attempts**, but only the **last result** counts
- More details later in the semester

Team Project

Project — basics

- Teams of **3–4 students** (4 recommended)
- One **large project** spanning the entire semester
- Simulating a small **startup** — real requirements, production quality
- AI usage is allowed and encouraged, but you are **responsible for code quality**

Project requirements

- Role-based **authorization** (user roles, access control)
- More than **10 database entities**
- **Back-office** admin interface
- **Email communication** (notifications, registration, etc.)
- **Component library** (shared UI elements)
- **Testing** — E2E and unit tests
- **Light and dark theme**
- **Mobile responsiveness**

Milestones

Week	Milestone	Focus
7	1st milestone	Design — ERD, wireframes, requirements
10	2nd milestone	Core implementation — DB, BE, FE
13	3rd milestone	Final review before defense
Exam period	Defense	Project presentation (~15 min)

Scoring

- **Milestones:** 40 points × number of team members
 - Points are **distributed within the team** as you see fit
- **Defense:** 30 points × number of team members
- Evaluated: functionality, code quality, collaboration

Project timeline

Period	Activity
Weeks 1–2	Forming teams
Week 3	Registering projects
Week 7	1st milestone
Week 10	2nd milestone
Week 13	3rd milestone
Exam period	Project defense

Recommendations

- Use **task management** tools (Jira, Trello, YouTrack)
- Divide work based on team members' **strengths**
- Use **Merge Requests** + code review
- Set up **CI/CD** pipelines (ESLint, Prettier)
- **Start early** — this project is significantly larger than previous years

Week 01 — What we do today

Today's goals

1. **Verify** your environment works (Node.js, pnpm, editor)
2. **Create** a new React project using Vite
3. **Write** simple React components with typed props
4. **Compose** components together into a UI

Step 1 — Install a Node version manager

Don't install Node.js directly. Use a **version manager** instead.

Tool	Platforms	Install
nvm	macOS, Linux	<code>curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh bash</code>
fnm	macOS, Linux, Windows	<code>curl -fsSL https://fnm.vercel.app/install bash</code>
asdf	macOS, Linux	<code>brew install asdf + asdf plugin add nodejs</code>

Windows users: **fnm** is the easiest option. Alternatively use **nvm-windows**.

Step 2 — Install Node.js

After installing your version manager, install **Node.js LTS**:

nvm:

```
nvm install --lts  
nvm use --lts
```

fnm:

```
fnm install --lts  
fnm use --lts
```

asdf:

```
asdf install nodejs latest  
asdf set --home nodejs latest
```

Step 3 — Install pnpm

We use **pnpm** as our package manager — faster, stricter, saves disk space.

```
corepack enable  
corepack prepare pnpm@latest --activate
```

Verify: `pnpm --version`

What is corepack? Built into Node.js — it manages package managers (pnpm, yarn) for you. No separate install needed.

Step 4 — Create a React project

```
pnpm create vite@latest my-app --template react-ts  
cd my-app  
pnpm install  
pnpm dev
```

Open `http://localhost:5173` in your browser.

Project structure tour

```
my-app/  
├── index.html          # Entry point  
├── package.json        # Dependencies & scripts  
├── tsconfig.json       # TypeScript config  
├── vite.config.ts      # Build tool config  
└── src/  
    ├── main.tsx        # App bootstrap  
    ├── App.tsx         # Root component  
    └── App.css          # Styles
```

Key files: `src/main.tsx` (entry) and `src/App.tsx` (your code goes here)

Why React?

Plain HTML + JS:

```
const list = document.getElementById('list');
data.forEach(item => {
  const li = document.createElement('li');
  li.textContent = item.name;
  list.appendChild(li);
});
```

React:

```
const List = ({ data }: { data: Item[] }) => (
  <ul>
    {data.map(item => <li key={item.id}>{item.name}</li>)}
  </ul>
);
```

A component is a function that returns UI

```
const Greeting = ({ name }: { name: string }) => {  
  return <h1>Hello, {name}</h1>;  
};
```

- **JSX** — HTML-like syntax inside JavaScript
- **Props** — data passed into a component
- **TS-lite** — simple types for now (`string`, `number`)

Exercise 1 — ProfileCard (15 min)

Create a `ProfileCard` component with props:

- `name: string`
- `bio: string`
- `avatarUrl: string`

Render **3 instances** with different data.

```
<ProfileCard  
  name="Alice"  
  bio="Frontend developer"  
  avatarUrl="https://i.pravatar.cc/150?img=1"  
>
```

Exercise 2 — UserList (20 min)

Create a `UserList` component that:

- Accepts an **array of users**
- Renders a `ProfileCard` for each using `.map()`

```
const UserList = ({ users }: { users: User[] }) => (  
  <div>  
    {users.map(user => (  
      <ProfileCard key={user.name} {...user} />  
    ))}  
  </div>  
);
```

Exercise 3 — Like button (20 min)

Add a **Like** button with a counter to `ProfileCard`.

```
import { useState } from 'react';

const ProfileCard = ({ name, bio, avatarUrl }: ProfileCardProps) => {
  const [likes, setLikes] = useState(0);

  return (
    <div>
      <img src={avatarUrl} alt={name} />
      <h2>{name}</h2>
      <p>{bio}</p>
      <button onClick={() => setLikes(likes + 1)}>
        Like {likes}
      </button>
    </div>
  );
};
```

Exercise 4 — SearchFilter (15 min, stretch)

Create a `SearchFilter` component that:

- Has a text input for filtering
- Filters the user list **in real time** by name

Hint: Use `useState` for the search text and `.filter()` on the users array.

Recap

- **Component** = a function that returns UI
- **Props** = data passed into a component
- **useState** = state inside a component
- **Composition** = components inside components

Next week: **Tailwind CSS** and **Shadcn** — making things look good

Questions?